

Machine Learning

Solutions to Tutorial 1, the Chapter 1 and 2 Exercises, and the Lecture 3 Review Questions

Detailed, step-by-step solutions in the notation of the lectures and of the one-read document.

How to use these solutions

Every problem is restated in a blue box, then solved. Each solution opens with a yellow box saying which concept the problem tests, so you can see why the question is asked, not only how to answer it. References like [one-read §2.2] point to the section of *Machine Learning: Lectures 1 to 4 in One Continuous Read* where the concept is explained. Numbers are worked to three or four significant digits so you can check your own calculations line by line. The professor's sheets ask you to try each question yourself first; the restated problem boxes let you do exactly that before reading the solution under it.

Contents

1	Tutorial 1: From learning problems to probabilistic classification	2
1.1	Problem 1: From a security objective to a learning problem	2
1.2	Problem 2: The optimal action depends on the loss	3
1.3	Problem 3: KNN inherits the geometry of the representation	5
1.4	Problem 4: Audit a generalisation claim	6
1.5	Problem 5: Capacity, regularisation and validation	8
1.6	Problem 6: Discriminative probabilities, decisions and prior shift	9
1.7	Problem 7: Generative classification, assumptions become visible	10
1.8	Problem 8: Detecting an attack from a sequence of flows	13
2	Exercises for Chapter 1: What does it mean to learn?	15
2.1	Exercise 1: Write the rules	15
2.2	Exercise 2: The knowledge engineer	16
2.3	Exercise 3: Two collections of photographs	16
2.4	Exercise 4: The mysterious shortcut	17
2.5	Exercise 5: Same world, different questions	17
2.6	Exercise 6: What exactly is being supervised?	18
2.7	Exercise 7: The robot that learns in the wrong world	18
2.8	Exercise 8: Design the experience	19
2.9	Exercise 9: Four ingredients, one system	19
2.10	Exercise 10: A claim worth questioning	20
3	Exercises for Chapter 2: From representations to decisions	20
3.1	Exercise 1: Representation and unavoidable ambiguity	20
3.2	Exercise 2: The loss changes the meaning of the best prediction	21

3.3	Exercise 3: Classification with a reject option	22
3.4	Exercise 4: Training loss and decision performance can disagree	23
3.5	Exercise 5: A nearest neighbour created by the units	24
3.6	Exercise 6: Local neighbourhoods in high dimension	24
3.7	Exercise 7: Perfect interpolation does not determine behaviour elsewhere	25
3.8	Exercise 8: Are parameters and hyperparameters intrinsic labels?	26
3.9	Exercise 9: Several empirical-risk minimisers	27
3.10	Exercise 10: Audit a misleading security experiment	28

4 Review questions for Lecture 3: From fitting to generalisation 28

1 Tutorial 1: From learning problems to probabilistic classification

1.1 Problem 1: From a security objective to a learning problem

Problem 1

A security operations centre receives completed network flows (packet times, addresses, ports, protocol, byte counts). A ground-truthing process labels each flow benign or attack using incident information or attack-scenario metadata; an attack label may mean “member of an attack episode” even when the flow itself looks benign. The deployed system must score a new flow and decide whether to raise an alert. (a) Identify the physical object, the representation x , the target Y , the model output, and the final action. (b) Some attack and benign flows have the same or very similar features. Why may a classifier using only these features be unable to separate them? Does this mean the labels are wrong? (c) Why are $\hat{p}_\theta(y = 1 | x)$ and the alert decision not the same object? (d) Give one feature available at decision time and one that would create target leakage. (e) Most attacks occurred at night, most benign flows by day. How can time of day become a shortcut? Why is this different from target leakage? (f) One cybersecurity task each for supervised, self-supervised and reinforcement learning, with the learning signal.

What this tests

The chain object $\rightarrow$ representation $\rightarrow$ model output $\rightarrow$ decision $\rightarrow$ action [one-read §1.3, 2.1, 2.2]; unavoidable error when the representation loses information; leakage versus shortcut [one-read §3.1, 3.6]; the three forms of experience [one-read §1.2].

(a) The pipeline has five distinct objects:

Physical object	the completed network flow itself: the actual exchange of packets between two endpoints.
Representation x	the feature vector computed from the flow record once it completes, e.g. duration, packets and bytes in each direction, protocol, destination port (encoded), TCP flags, packet-timing statistics.
Target Y	the label from the ground-truthing process, $Y \in \{0 = \text{benign}, 1 = \text{attack}\}$.
Model output	a score $s_\theta(x)$ or, after the sigmoid, a posterior estimate $\hat{p}_\theta(y = 1 x)$.
Final action	raise an alert or not (possibly a third action, send to an analyst for review).

(b) A classifier that uses only x is a function $h(x)$: it gives *one* answer for each feature vector. If an attack flow and a benign flow have the same x , h labels both the same way and is wrong on one of them. In probability terms, for such an x the posterior $P(Y = 1 | X = x)$ is strictly between 0 and 1, and even the Bayes classifier errs with probability $\min\{P(Y = 1 | x), P(Y = 0 | x)\} > 0$ there [one-read §2.1, mastery box].

This does *not* mean the labels are wrong. The label is defined by episode membership: a flow that is part of an attack episode is “attack” even if, on its own, it is an ordinary-looking connection (for instance a reconnaissance HTTP request). The label can be perfectly correct *as defined*; what is missing is information in x , such as the other flows from the same host or the time context. Label noise is possible too, but similar features with different labels are not evidence of it.

(c) $\hat{p}_{\hat{\theta}}(y = 1 \mid x)$ is a *prediction*: a number that summarises what the features say about the outcome. The alert is a *decision*: the result of applying a decision rule to that number, which also needs the costs of a false alarm and of a missed attack (and possibly the analysts’ capacity). The same probability, say 0.3, should raise an alert in a SOC where a miss costs much more than a false alarm and not in one where alert fatigue dominates. Keeping them separate also lets you change the decision rule without retraining the model [one-read §1.3, 2.2].

(d)

- *Available at decision time*: the flow’s duration, packet count, byte count, destination port, protocol. All are known the moment the flow completes.
- *Target leakage*: any field produced by the investigation that created the label, for example the incident ticket ID, the attack-scenario name from the metadata, or “host was quarantined afterwards”. It predicts the label almost perfectly in the dataset and does not exist when a new flow has to be scored.

(e) If most attacks in the collected data happen at night, “hour of day” predicts the label well in training, so the model can use it instead of learning what attack traffic looks like. At deployment, a daytime attack is missed and night-time benign jobs (backups, updates) raise alerts.

This is a *shortcut*, not leakage. Time of day *is* available when a new flow is scored, so using it is not cheating about what is known. The problem is that the correlation was created by how the data were collected and is not stable. Leakage uses information that would *not* be available at decision time at all [one-read §3.1, 3.6]. The remedies differ too: leakage is fixed by removing the field; a shortcut is fixed by collecting data where the nuisance factor varies (attacks by day, benign traffic by night) and testing on such data.

(f)

Paradigm	Task	Learning signal
Supervised	classify flows as benign or attack	the ground-truth label of each training flow
Self-supervised	learn a representation of traffic by predicting a masked field of a flow, or the next flow of a host from the previous ones	the hidden part of the traffic itself; no human label
Reinforcement	an automated response agent chooses block, rate-limit, quarantine or do nothing, in a simulated network	a reward: attack contained, minus the cost of disrupting legitimate users; the agent’s actions change what happens next

1.2 Problem 2: The optimal action depends on the loss

Problem 2

For a flow with features x , let $p = p(y = 1 \mid x)$ (class 1 = attack). A false alert costs $C_{FP} = 2$, a missed attack $C_{FN} = 8$; correct decisions cost 0. (a) Derive the conditional risk of declaring attack and of declaring benign, and the optimal probability threshold. (b) Apply the rule to posteriors 0.08, 0.35, 0.85. (c) Human review costs 0.9 whatever the true class and then resolves the case correctly. Derive the regions for benign, review, attack. (d) Why is changing the threshold for error costs different from correcting a model for class imbalance?

What this tests

Conditional risk and the Bayes decision rule [one-read §2.2]: write the expected cost of each action as a function of p , then pick the smallest. With three actions the smallest switches twice.

(a) The conditional risk of an action is its expected cost given x , averaging over the two possible truths:

$$\text{declare attack} \quad R(\text{attack} | x) = \underbrace{C_{FP}}_{\text{if benign}} \cdot \underbrace{(1-p)}_{P(\text{benign})} + 0 \cdot p = 2(1-p),$$

$$\text{declare benign} \quad R(\text{benign} | x) = \underbrace{C_{FN}}_{\text{if attack}} \cdot \underbrace{p}_{P(\text{attack})} + 0 \cdot (1-p) = 8p.$$

Declare attack when it is cheaper: $2(1-p) < 8p \iff 2 < 10p \iff p > 0.2$. In general $p > C_{FP}/(C_{FP} + C_{FN}) = 2/10$. The threshold is far below 0.5 because a miss is four times as costly as a false alarm.

(b)

p	$R(\text{attack}) = 2(1-p)$	$R(\text{benign}) = 8p$	decision
0.08	1.84	0.64	benign
0.35	1.30	2.80	attack
0.85	0.30	6.80	attack

Note the flow at 0.35: it is more likely benign than attack, and we still alert, because missing it would be expensive.

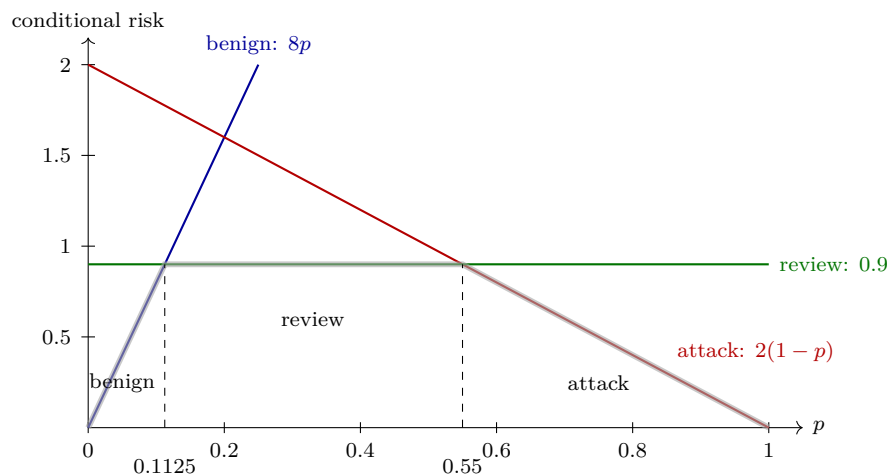
(c) Add a third action with constant risk $R(\text{review} | x) = 0.9$. Each action is optimal where its risk is the smallest of the three:

$$\text{benign optimal} \quad 8p \leq 0.9 \text{ and } 8p \leq 2(1-p) \iff p \leq 0.1125 \text{ and } p \leq 0.2 \iff p \leq 0.1125,$$

$$\text{attack optimal} \quad 2(1-p) \leq 0.9 \text{ and } 2(1-p) \leq 8p \iff p \geq 0.55 \text{ and } p \geq 0.2 \iff p \geq 0.55,$$

$$\text{review optimal} \quad 0.1125 < p < 0.55.$$

Check: at $p = 0.3$ the three risks are benign 2.4, attack 1.4, review 0.9, so review wins. The review band exists because, without review, the lower envelope $\min\{8p, 2(1-p)\}$ peaks at $p = 0.2$ with value $1.6 > 0.9$. With review, the three flows of (b) become: 0.08 benign, 0.35 review, 0.85 attack.



The grey path is the lower envelope: the risk of the best action at each p .

(d) The cost threshold acts at the *decision* stage: it assumes the posterior is right and asks what to do with it. A class-imbalance correction acts at the *estimation* stage: either the minority class has too few examples for the posterior to be estimated well (weighting or resampling helps the fit), or the class frequencies at deployment differ from training (prior shift, which changes the posterior itself) [one-read §4.7]. Moving the threshold cannot repair a badly estimated posterior, and reweighting the training data does not encode what a missed attack costs. Different questions, different evidence: costs come from the operations team, prevalence from data about the deployment environment.

1.3 Problem 3: KNN inherits the geometry of the representation

Problem 3

Query flow $x = (50, 10)$: duration 50 ms, 10 packets. Labelled flows $x_A = (60, 100)$ and $x_B = (400, 12)$. (a) Nearest flow with raw Euclidean distance. (b) Scale duration by 100 ms and packet count by 20 packets; nearest flow after scaling. (c) With z the scaled features, A = attack, B = benign: derive and plot the 1-NN boundary between the two training flows; mark the query and its class. (d) With $d_w^2(z, z') = w(z_1 - z'_1)^2 + (z_2 - z'_2)^2$, $w > 0$: find the critical w where the nearest flow changes; which flow is nearest on each side; interpret; why choose w by validation?

What this tests

KNN has no notion of “similar” except the distance you give it, so units and weights decide the answer [one-read §2.4]. The 1-NN boundary between two points is their perpendicular bisector.

(a) Differences from the query: to A (10, 90), to B (350, 2).

$$d(x, x_A) = \sqrt{10^2 + 90^2} = \sqrt{8200} \approx 90.55, \quad d(x, x_B) = \sqrt{350^2 + 2^2} = \sqrt{122504} \approx 350.01.$$

A is nearest. Duration in milliseconds has the larger numbers, so it dominates: B is “far” because of 350 ms, even though its packet count (12) is almost identical to the query’s (10).

(b) Scaled features $z = (\text{duration}/100, \text{packets}/20)$: query $z = (0.5, 0.5)$, $z_A = (0.6, 5.0)$, $z_B = (4.0, 0.6)$.

$$d(z, z_A) = \sqrt{0.1^2 + 4.5^2} = \sqrt{20.26} \approx 4.50, \quad d(z, z_B) = \sqrt{3.5^2 + 0.1^2} = \sqrt{12.26} \approx 3.50.$$

Now B is nearest. Nothing about the flows changed; only the units, and with them the geometry.

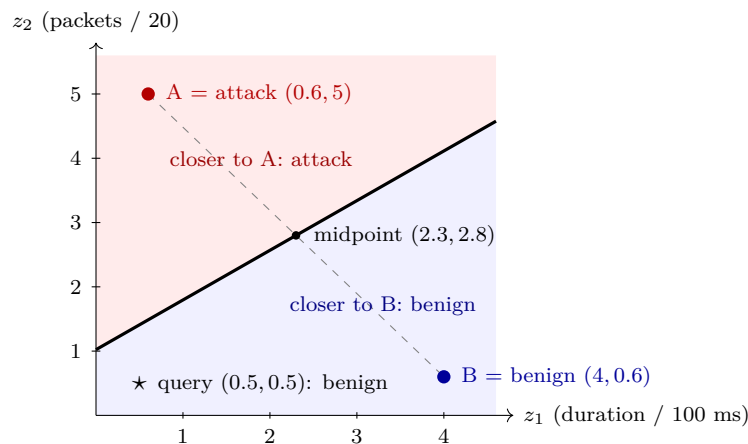
(c) The 1-NN rule predicts the label of whichever training point is closer, so the boundary is the set of points equidistant from z_A and z_B :

$$\|z - z_A\|^2 = \|z - z_B\|^2 \iff -2z^\top z_A + \|z_A\|^2 = -2z^\top z_B + \|z_B\|^2 \iff 2z^\top (z_B - z_A) = \|z_B\|^2 - \|z_A\|^2.$$

(The $\|z\|^2$ terms cancel, which is why the boundary is a straight line.) With $z_B - z_A = (3.4, -4.4)$, $\|z_B\|^2 = 16 + 0.36 = 16.36$, $\|z_A\|^2 = 0.36 + 25 = 25.36$:

$$2(3.4z_1 - 4.4z_2) = -9 \iff 6.8z_1 - 8.8z_2 = -9 \iff z_2 = 0.7727z_1 + 1.0227.$$

It passes through the midpoint (2.3, 2.8) and is perpendicular to the segment AB. Points above the line are closer to A (attack), points below closer to B (benign). The query (0.5, 0.5): the line at $z_1 = 0.5$ is at $z_2 = 1.409 > 0.5$, so the query is below it: **predicted benign**, consistent with (b).



(d) Squared weighted distances from the query:

$$d_w^2(z, z_A) = w(0.1)^2 + (4.5)^2 = 0.01w + 20.25, \quad d_w^2(z, z_B) = w(3.5)^2 + (0.1)^2 = 12.25w + 0.01.$$

They are equal when $0.01w + 20.25 = 12.25w + 0.01$, i.e. $20.24 = 12.24w$:

$$w^* = \frac{20.24}{12.24} \approx 1.654.$$

For $w < 1.654$, B is nearer (benign); for $w > 1.654$, A is nearer (attack). (Check at $w = 1$: A gives 20.26, B gives 12.26, so B, as in (b).)

Interpretation. The query differs from A mainly in packet count (4.5 scaled units) and from B mainly in duration (3.5 units). w says how much a unit of duration difference counts relative to a unit of packet-count difference. Weighting duration heavily makes B look far, so A wins. The label depends on a modelling choice about which feature matters.

Why validation. w is a hyperparameter: it is not estimated by the 1-NN rule. Choosing it by looking at this query, or at test answers, would tune the metric to the cases you evaluate on. The right w is the one that classifies held-out flows best, so it must be selected on validation data and then assessed once on the test set [one-read §2.7].

1.4 Problem 4: Audit a generalisation claim

Problem 4

A team collects 200,000 flow records; one attack episode can generate many nearly duplicate flows. They standardise all records before splitting, randomly split rows into training and test, try 100 model and feature configurations, select the one with the highest test accuracy, and report that same accuracy. (a) Identify every mechanism that makes the reported performance too optimistic. (b) Propose a training, validation and test protocol for: i. new flows from the same simulated process; ii. later traffic under drift; iii. new attack episodes. (c) Test set: 10,000 flows, 100 attacks; $TP = 70$, $FP = 180$, $FN = 30$, $TN = 9720$. Compute accuracy, precision, recall, F_1 . (d) Compare with “everything benign”. Why is accuracy insufficient? (e) Where must scaling, missing-value treatment and feature selection be fitted in a cross-validation pipeline?

What this tests

Every subset may influence only what it is allowed to [one-read §2.7]; data-dependent preprocessing belongs to training; selection on a set makes that set optimistic [one-read §3.1, mastery boxes]; splits must mimic deployment novelty [one-read §3.6].

(a)

1. *Standardising before splitting.* The means and standard deviations are computed with the test records, so test information enters the representation (preprocessing leakage). For standardisation the effect is usually small, but it breaks the rule, and the same pattern with imputation or feature selection can be large.
2. *Random row split with near-duplicates.* Flows from one episode land on both sides, so the test set contains near-copies of training flows. The model is rewarded for recognising the episodes it has already seen, not for detecting new ones (group leakage).
3. *Selecting among 100 configurations by test accuracy.* The test set has become a validation set. The best of 100 noisy scores is optimistic even if all configurations are equally good: $\mathbb{E}[\min_j \hat{R}(\hat{f}_j)] \leq \min_j R(\hat{f}_j)$ [one-read §3.1], the winner's curse.
4. *Reporting the selection score as final.* No untouched data remains to estimate the chosen model's performance without bias.
5. *Accuracy as the only metric.* Not an optimism of the estimate but of the impression: with rare attacks, accuracy is dominated by benign flows and can be high while most attacks are missed (see (c), (d)).

(b) In every protocol, all data-dependent preprocessing is fitted on the training part only, hyperparameters and the threshold are chosen on validation, and the test set is used once.

- i. *Same simulated process.* Remove exact and near duplicates first, then a stratified random split of rows (e.g. 60/20/20) is appropriate, since deployment flows are like independent draws from the same process.
- ii. *Later traffic, possible drift.* A temporal split: train on the earliest period, validate on the next, test on the latest, with a gap between periods at least as long as any feature window. This measures how the model ages.
- iii. *New attack episodes.* A grouped split by episode: every flow of an episode is in exactly one subset (benign traffic grouped by host or time block too). For selection, use grouped K -fold within the development data. To measure novelty, hold out entire attack *types* in the test set.

(c)

$$\begin{aligned}
 \text{accuracy} \quad & \frac{TP + TN}{\text{all}} = \frac{70 + 9720}{10000} = 0.979 \\
 \text{precision} \quad & \frac{TP}{TP + FP} = \frac{70}{250} = 0.28 \\
 \text{recall} \quad & \frac{TP}{TP + FN} = \frac{70}{100} = 0.70 \\
 F_1 \quad & \frac{2 \times 0.28 \times 0.70}{0.28 + 0.70} = \frac{0.392}{0.98} = 0.40
 \end{aligned}$$

Also useful: specificity $9720/9900 = 0.982$, false-positive rate $180/9900 = 0.018$, i.e. about 1,818 false alarms per 100,000 benign flows.

(d) “Everything benign” gives $TP = 0$, $FN = 100$, $FP = 0$, $TN = 9900$: accuracy $9900/10000 = 0.99$, *higher* than the real classifier, with recall 0 and precision undefined (0/0). Accuracy rewards the useless rule because it is a weighted average dominated by the 99% benign flows. The real classifier catches 70% of attacks at the price of 180 false alarms, about 2.6 false alarms per true detection; whether that is acceptable is a question about costs and analyst capacity that accuracy cannot even express.

(e) Inside each cross-validation split, the scaler (means and variances), the imputer (fill values), and the feature selector (scores, selected subset) are fitted on the $K-1$ training folds only and then *applied* unchanged to the held-out fold. After selection, the whole pipeline is refitted on all development data, and applied unchanged to the test set and at deployment [one-read §3.6]. In practice: put them in one pipeline object and cross-validate the pipeline, never the model alone.

1.5 Problem 5: Capacity, regularisation and validation

Problem 5

Three nested families $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \mathcal{H}_3$ fitted to one training sample: training errors 0.20, 0.08, 0.00; validation errors 0.22, 0.11, 0.19. (a) Why can the minimum training error not increase from $\mathcal{H}_1$ to $\mathcal{H}_3$? (b) Which family do you select? Diagnose the other two. (c) With much more training data, the validation error of $\mathcal{H}_3$ drops to 0.10. Why can capacity not be judged without the sample size? (d) Two configurations have the same population error $R = 0.20$; validation errors $\hat{R}_i = R + \varepsilon_i$ with independent $\varepsilon_i \sim N(0, \sigma^2)$, $\sigma = 0.02$. The smaller is selected. Using $\min(a, b) = \frac{a+b-|a-b|}{2}$ and $\mathbb{E}|\varepsilon_1 - \varepsilon_2| = 2\sigma/\sqrt{\pi}$, derive the expected reported validation error and interpret.

What this tests

Nested families and monotone training error; reading training and validation error together; capacity is relative to N [one-read §3.2]; selection makes the winner's score optimistic [one-read §3.5].

(a) Every function available in $\mathcal{H}_1$ is also in $\mathcal{H}_2$ and $\mathcal{H}_3$. The best training fit in $\mathcal{H}_3$ is a minimum over a *larger* set, which contains the best fit of $\mathcal{H}_1$ as one candidate, so it can only be as small or smaller:

$$\min_{f \in \mathcal{H}_3} \hat{R}_{\text{train}}(f) \leq \min_{f \in \mathcal{H}_2} \hat{R}_{\text{train}}(f) \leq \min_{f \in \mathcal{H}_1} \hat{R}_{\text{train}}(f).$$

(This assumes the optimiser actually finds the minimum in each family.)

(b) Select $\mathcal{H}_2$: it has the smallest validation error, 0.11.

- $\mathcal{H}_1$: training 0.20, validation 0.22. Both high and close together: the family is too restrictive to capture the structure. **Underfitting** (high bias).
- $\mathcal{H}_3$: training 0.00, validation 0.19. It interpolates the sample and the gap is 0.19: it has fitted accidental variation. **Excessive capacity for this sample**, i.e. high estimation variance (overfitting).
- $\mathcal{H}_2$: training 0.08, validation 0.11, small gap and lowest validation error: the best trade-off here.

(c) Variance shrinks as N grows: with more representative data, fewer functions in $\mathcal{H}_3$ fit the sample while behaving badly elsewhere, so the fitted function becomes stable and its validation error drops to 0.10, now the best of the three. The same family that overfitted a small sample is well constrained by a large one. “Too complex” is therefore not a property of $\mathcal{H}_3$ alone but of $\mathcal{H}_3$ *relative to the evidence available*; the right capacity grows with N [one-read §3.2].

(d) The reported value is the smaller of the two validation errors.

$$\begin{aligned} \mathbb{E}[\min(\hat{R}_1, \hat{R}_2)] &= \mathbb{E}\left[\frac{\hat{R}_1 + \hat{R}_2 - |\hat{R}_1 - \hat{R}_2|}{2}\right] && \text{(the identity for the minimum)} \\ &= \frac{\mathbb{E}\hat{R}_1 + \mathbb{E}\hat{R}_2}{2} - \frac{\mathbb{E}|\varepsilon_1 - \varepsilon_2|}{2} && (\hat{R}_1 - \hat{R}_2 = \varepsilon_1 - \varepsilon_2) \\ &= \frac{R + R}{2} - \frac{2\sigma/\sqrt{\pi}}{2} = R - \frac{\sigma}{\sqrt{\pi}}. \end{aligned}$$

(Where the given constant comes from: $\varepsilon_1 - \varepsilon_2 \sim N(0, 2\sigma^2)$, and $\mathbb{E}|Z| = \tau\sqrt{2/\pi}$ for $Z \sim N(0, \tau^2)$; with $\tau = \sigma\sqrt{2}$ this is $2\sigma/\sqrt{\pi}$.)

Numerically: $0.20 - 0.02/1.7725 = 0.20 - 0.0113 \approx 0.189$.

Interpretation. Neither configuration is better, yet the reported score is on average 0.011 below the true error. Selecting the minimum picks whichever model got luckier noise on this particular validation set. With more configurations the minimum of more noisy estimates is even more optimistic. That is why the selected model must be re-measured on a test set that played no part in the choice.

1.6 Problem 6: Discriminative probabilities, decisions and prior shift

Problem 6

A binary discriminative model produces a score $s_{\hat{\theta}}(x)$ and $r_t(x) = \hat{p}_{\hat{\theta}}(y = 1 \mid x) = 1/(1 + e^{-s_{\hat{\theta}}(x)})$. (a) For $s = 1.2$, compute the posterior, and the cross-entropy loss if the true label is 1. (b) Why is minimising empirical cross-entropy equivalent to maximising the conditional likelihood under an i.i.d. assumption? (c) With $C_{FP} = 4$, $C_{FN} = 16$: cost-sensitive threshold and decision for this flow. (d) For $s(x) = -3 + 2x_1 + x_2$, plot the boundary with threshold 0.5 and with the threshold of (c); indicate the attack region; explain the movement. (e) Training prior $\pi_t = 0.20$, deployment prior $\pi_d = 0.05$, calibrated $r_t(x) = 0.60$. Compute $r_d(x)$ with the prior-shift formula. (f) Distinguish this correction from a cost-based threshold. (g) With a very small minority sample, why may weighting or resampling help even without prior shift?

What this tests

Score $\rightarrow$ sigmoid $\rightarrow$ probability $\rightarrow$ decision [one-read §4.2]; cross-entropy as negative log-likelihood; a probability threshold is a score threshold, so the boundary moves in parallel; prior shift changes the probability, costs change the action [one-read §4.7].

(a) $e^{-1.2} = 0.3012$, so

$$r_t = \sigma(1.2) = \frac{1}{1 + 0.3012} = 0.7685, \quad \ell_{CE} = -\log r_t = -\ln 0.7685 = 0.2633.$$

(Equivalently $\ell_{CE} = \ln(1 + e^{-s})$ when $y = 1$.)

(b) For a binary label, the model's probability of the observed label is $p_{\theta}(y \mid x) = r_{\theta}(x)^y (1 - r_{\theta}(x))^{1-y}$: it equals r if $y = 1$ and $1 - r$ if $y = 0$. If the labels are conditionally independent given the inputs (the i.i.d. assumption), the probability of all observed labels is the product

$$\begin{aligned} \text{conditional likelihood} \quad L_{\text{cond}}(\theta) &= \prod_{i=1}^N p_{\theta}(y^{(i)} \mid x^{(i)}) \\ \text{its negative log} \quad -\log L_{\text{cond}}(\theta) &= -\sum_{i=1}^N \left[y^{(i)} \log r_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - r_{\theta}(x^{(i)})) \right] \\ &= N \hat{R}_{CE}(\theta). \end{aligned}$$

The logarithm is increasing, so maximising L and maximising $\log L$ pick the same θ ; multiplying by $-1/N$ turns “maximise” into “minimise” without changing the optimiser. Hence $\arg \max_{\theta} L_{\text{cond}} = \arg \min_{\theta} \hat{R}_{CE}$.

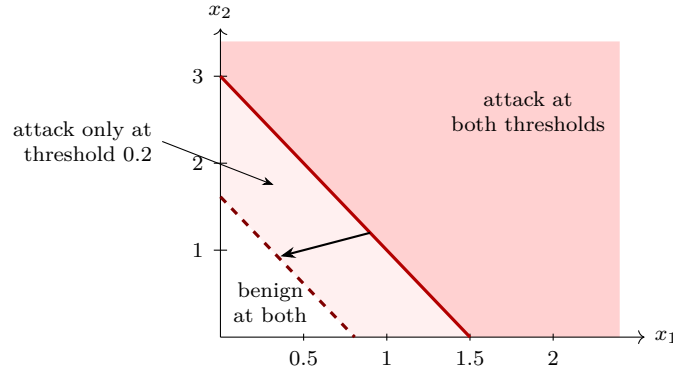
(c) Threshold $\tau = C_{FP}/(C_{FP} + C_{FN}) = 4/20 = 0.2$. Since $0.7685 > 0.2$, declare **attack**. In score terms, $\sigma(s) > \tau \iff s > \log \frac{\tau}{1-\tau} = \ln 0.25 = -1.386$, and $1.2 > -1.386$.

(d) The sigmoid is increasing, so a probability threshold τ is a score threshold $\log \frac{\tau}{1-\tau}$:

$$\text{threshold 0.5} \quad s > 0 \iff 2x_1 + x_2 > 3,$$

$$\text{threshold 0.2} \quad s > \ln 0.25 = -1.386 \iff 2x_1 + x_2 > 3 - 1.386 = 1.614.$$

Both boundaries are lines with the same normal vector $(2, 1)$: they are *parallel*. The second is shifted toward the origin by $1.386/\sqrt{2^2 + 1^2} = 0.62$ units. The attack region (the side where $2x_1 + x_2$ is large) grows.



solid line: threshold 0.5, boundary $2x_1 + x_2 = 3$ dashed line: threshold 0.2, boundary $2x_1 + x_2 = 1.614$
the arrow shows the parallel shift of 0.62 units toward the benign side

Why it moves this way. Lowering the probability threshold means alerting on weaker evidence, i.e. on smaller scores. Because the score is linear in x , “score above a smaller constant” is a half-plane bounded by a parallel line closer to the benign side. The direction of the boundary is fixed by the score family; the threshold only slides it [one-read §4.2].

(e) With $\pi_d/\pi_t = 0.05/0.20 = 0.25$ and $(1 - \pi_d)/(1 - \pi_t) = 0.95/0.80 = 1.1875$:

$$r_d(x) = \frac{0.25 \times 0.60}{0.25 \times 0.60 + 1.1875 \times 0.40} = \frac{0.15}{0.15 + 0.475} = \frac{0.15}{0.625} = 0.24.$$

Check with odds. Training odds $0.6/0.4 = 1.5$; prior-odds ratio $\frac{0.05/0.95}{0.20/0.80} = 0.2105$; new odds $1.5 \times 0.2105 = 0.316$; $r_d = 0.316/1.316 = 0.24$. Attacks are four times rarer in deployment, so the same features are much weaker evidence of an attack. With the cost threshold 0.2 of (c) this flow still alerts ($0.24 > 0.2$); with a 0.5 threshold it would have flipped from attack (0.60) to benign (0.24).

(f) The prior-shift correction changes the *probability*, because the world changed: the likelihood ratio $p(x | 1)/p(x | 0)$ is the same, but attacks are rarer, so the posterior must be updated. The cost threshold changes the *action*, because the consequences of the two errors differ; it assumes the probability is already right. Apply them in that order: correct the posterior for the deployment prior, then threshold the corrected posterior using the costs. They need different evidence (prevalence estimates versus cost estimates), even though algebraically both end up as a threshold on r_t .

(g) With very few attack examples, they contribute almost nothing to the training objective, which is an average dominated by benign flows. The fit can then place the boundary wherever it suits the benign data and treat the few attacks as noise, so the attack region is poorly estimated (high variance, or simply ignored). Weighting attack examples more, or resampling them, makes the optimiser take them seriously and can improve where the boundary lies and how flows are ranked. The price: the fitted probabilities now correspond to an artificial, reweighted prior, so they are no longer calibrated. Before using a cost threshold they must be recalibrated, or corrected with the prior-shift formula using the effective training prior [one-read §4.7].

1.7 Problem 7: Generative classification, assumptions become visible

Problem 7, Part I (Gaussian class models)

$x = (x_1, x_2)^\top$, two standardised features; $p_\theta(x | y = c) = \mathcal{N}(\mu_c, \Sigma)$ with $\mu_0 = (0, 0)^\top$, $\mu_1 = (2, 2)^\top$, $\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$, $|\rho| < 1$; priors $\pi_0 = 0.75$, $\pi_1 = 0.25$. (a) Show $\Sigma^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}$. Derive the LDA score $s_c(x) = x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c$ from the Gaussian log posterior; explain why the boundary is linear. (b) $\rho = 0.5$: explicit s_0, s_1 ; boundary as an equation in $x_1 + x_2$; plot with the means. (c) $x = (1.25, 1.25)^\top$: predicted class and $p(y = 1 | x)$. (d) Repeat (b), (c) with equal

priors; explain how the prior moves the boundary.

What this tests

Generative scores and Bayes' rule; the LDA derivation (which terms cancel and why); priors shift the line, covariance sets its direction [one-read §4.4].

(a) *Inverse.* For a 2×2 matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ the inverse is $\frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$; here $ad - bc = 1 - \rho^2 > 0$. Check by multiplying:

$$\begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \frac{1}{1 - \rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix} = \frac{1}{1 - \rho^2} \begin{pmatrix} 1 - \rho^2 & -\rho + \rho \\ \rho - \rho & -\rho^2 + 1 \end{pmatrix} = I.$$

LDA score. Bayes' rule gives $\log p(y = c | x) = \log \pi_c + \log \mathcal{N}(x; \mu_c, \Sigma) - \log p(x)$, and $\log p(x)$ is the same for both classes, so we compare $s_c(x) = \log \pi_c + \log \mathcal{N}(x; \mu_c, \Sigma)$. Write out the log density and expand the quadratic:

$$\begin{aligned} \log \mathcal{N}(x; \mu_c, \Sigma) &= -\frac{d}{2} \log 2\pi - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (x - \mu_c)^\top \Sigma^{-1} (x - \mu_c), \\ (x - \mu_c)^\top \Sigma^{-1} (x - \mu_c) &= x^\top \Sigma^{-1} x - 2x^\top \Sigma^{-1} \mu_c + \mu_c^\top \Sigma^{-1} \mu_c, \end{aligned}$$

using $x^\top \Sigma^{-1} \mu_c = \mu_c^\top \Sigma^{-1} x$, which holds because Σ^{-1} is symmetric. Since Σ is shared, $-\frac{d}{2} \log 2\pi$, $-\frac{1}{2} \log |\Sigma|$ and $-\frac{1}{2} x^\top \Sigma^{-1} x$ are identical for both classes and can be dropped. What remains is

$$s_c(x) = x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c.$$

Why linear. $s_1(x) - s_0(x) = x^\top \Sigma^{-1} (\mu_1 - \mu_0) + \text{constant}$, which is linear in x . The boundary $s_1(x) = s_0(x)$ is the set where a linear function equals zero: a straight line (a hyperplane in general). The quadratic term $x^\top \Sigma^{-1} x$, the only thing that could curve it, cancelled because the covariance is shared.

(b) With $\rho = 0.5$: $1 - \rho^2 = 0.75$, so $\Sigma^{-1} = \frac{1}{0.75} \begin{pmatrix} 1 & -0.5 \\ -0.5 & 1 \end{pmatrix} = \begin{pmatrix} 4/3 & -2/3 \\ -2/3 & 4/3 \end{pmatrix}$.

Class 0: $\mu_0 = 0$ kills the first two terms, so $s_0(x) = \log 0.75 \approx -0.2877$ (a constant).

Class 1: $\Sigma^{-1} \mu_1 = (\frac{4}{3} \cdot 2 - \frac{2}{3} \cdot 2, -\frac{2}{3} \cdot 2 + \frac{4}{3} \cdot 2) = (\frac{4}{3}, \frac{4}{3})$, so $x^\top \Sigma^{-1} \mu_1 = \frac{4}{3}(x_1 + x_2)$ and $\mu_1^\top \Sigma^{-1} \mu_1 = 2 \cdot \frac{4}{3} + 2 \cdot \frac{4}{3} = \frac{16}{3}$:

$$s_1(x) = \frac{4}{3}(x_1 + x_2) - \frac{8}{3} + \log 0.25.$$

Boundary. $s_1 = s_0 \iff \frac{4}{3}(x_1 + x_2) = \frac{8}{3} + \log 0.75 - \log 0.25 = \frac{8}{3} + \log 3$:

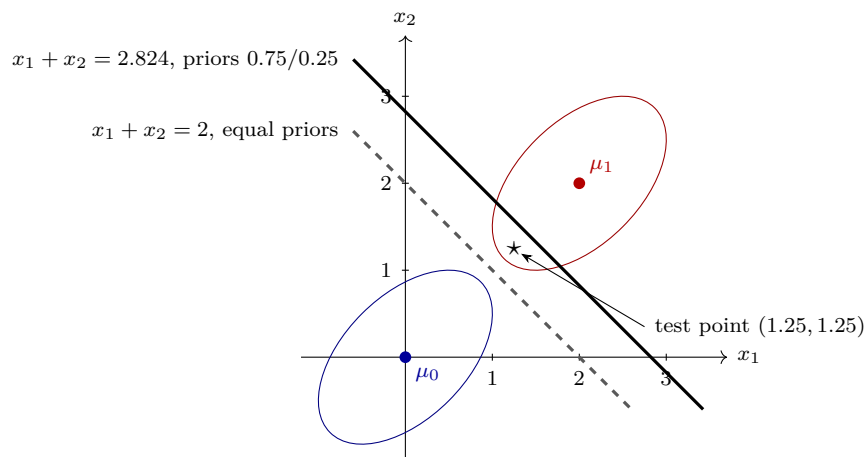
$$x_1 + x_2 = 2 + \frac{3}{4} \ln 3 \approx 2 + 0.824 = 2.824.$$

(c) For $(1.25, 1.25)$, $x_1 + x_2 = 2.5 < 2.824$: **class 0**. Scores: $s_0 = \ln 0.75 = -0.2877$; $s_1 = \frac{4}{3}(2.5) - \frac{8}{3} + \ln 0.25 = 3.3333 - 2.6667 - 1.3863 = -0.7196$. Normalising,

$$p(y = 1 | x) = \frac{e^{s_1}}{e^{s_0} + e^{s_1}} = \frac{0.4869}{0.75 + 0.4869} = 0.394.$$

(d) Equal priors: both scores get the same $\log 0.5$, which cancels. $s_1 = s_0 \iff \frac{4}{3}(x_1 + x_2) = \frac{8}{3} \iff x_1 + x_2 = 2$, the line through the midpoint $(1, 1)$ of the two means. The test point has $x_1 + x_2 = 2.5 > 2$: **class 1**, with $s_1 - s_0 = \frac{4}{3}(2.5) - \frac{8}{3} = \frac{2}{3}$ and $p(y = 1 | x) = 1/(1 + e^{-2/3}) = 0.661$.

How the prior moves the boundary. The class models, hence the direction of the line, are unchanged; only its offset moves. With $\pi_1 = 0.25$, class 1 is rarer, so the line moves from $x_1 + x_2 = 2$ toward μ_1 by $\frac{3}{4} \ln 3 = 0.824$ in the $x_1 + x_2$ coordinate (a perpendicular distance of $0.824/\sqrt{2} = 0.58$): to be classified as the rare class, a point must be clearly closer to it. The test point lies between the two lines, which is exactly why its prediction flips.



Ellipses: one-standard-deviation contours of each class Gaussian ($\rho = 0.5$ stretches them along $(1, 1)$).

(Here $\mu_1 - \mu_0 = (2, 2)$ happens to lie along an eigenvector of Σ , so the boundary is perpendicular to the segment between the means. With a general covariance the boundary is perpendicular to $\Sigma^{-1}(\mu_1 - \mu_0)$ instead, which need not be the direction joining the means.)

Problem 7, Part II (Naive Bayes)

$x_1 = 1$ if destination port is 22; $x_2 = 1$ if the duration is below a threshold τ .

	$p(x_1 = 1 \mid y = c)$	$p(x_2 = 1 \mid y = c)$	π_c
attack	0.60	0.70	0.10
benign	0.05	0.20	0.90

(a) Unnormalised scores and attack posterior for $x_1 = x_2 = 1$. (b) State the assumption; why might it fail for flows? (c) How can correlated features repeat evidence and produce overconfident probabilities? (d) A port value never seen in training appears in validation: how do an “other” category and additive smoothing prevent a zero likelihood? (e) One reason to prefer each route here.

(a)

$$\text{attack: } \pi_1 p(x_1=1 \mid 1) p(x_2=1 \mid 1) = 0.10 \times 0.60 \times 0.70 = 0.042,$$

$$\text{benign: } \pi_0 p(x_1=1 \mid 0) p(x_2=1 \mid 0) = 0.90 \times 0.05 \times 0.20 = 0.009,$$

$$p(\text{attack} \mid x) = \frac{0.042}{0.042 + 0.009} = 0.824.$$

(b) Conditional independence given the class: $p(x_1, x_2 \mid y = c) = p(x_1 \mid y = c) p(x_2 \mid y = c)$. Once you know whether the flow is an attack, knowing it is on port 22 should tell you nothing more about whether it is short. For flows this is doubtful: an SSH brute-force attack produces many short, failed connections *to port 22*, so within the attack class “port 22” and “short” tend to occur together because one behaviour causes both. Within benign traffic, port-22 sessions are often long interactive logins, a dependence in the opposite direction.

(c) Read Naive Bayes in odds form. The posterior odds are the prior odds times one likelihood ratio per feature:

$$\frac{p(\text{attack} \mid x)}{p(\text{benign} \mid x)} = \underbrace{\frac{0.10}{0.90}}_{\text{prior odds}} \times \underbrace{\frac{0.60}{0.05}}_{=12, \text{ port}} \times \underbrace{\frac{0.70}{0.20}}_{=3.5, \text{ duration}} = 4.67 \Rightarrow p = \frac{4.67}{5.67} = 0.824.$$

Each factor is counted as an *independent* piece of evidence. If the two features largely express the same fact, the same evidence is multiplied in twice. Extreme case: if x_2 were an exact copy of

x_1 , Naive Bayes would compute odds $\frac{1}{9} \times 12 \times 12 = 16$, i.e. $p = 0.94$, while the truth, using the information once, is $\frac{1}{9} \times 12 = 1.33$, i.e. $p = 0.57$. The ranking may still be right (a higher score still means more suspicious), but the probabilities are too extreme. Removing redundant features improves calibration.

(d) Without precautions, a port value v never seen in training has estimated probability $\hat{p}(x_j = v | y = c) = 0$ in every class, so every class score contains a factor 0, both unnormalised posteriors are 0, and the posterior is $0/0$, undefined. If it was seen in only one class, the other class gets probability 0 and the single unseen value decides the case with certainty. Two fixes work together:

- *An “other” category.* During training preprocessing, map rare ports (seen fewer than m times) to a level called *other*, so it has training counts and is one of the K_j levels. At validation, test and deployment, map every port not in the training vocabulary to *other*. Every possible input now has a defined level.
- *Additive smoothing.* Estimate $\hat{p}(x_j = v | y = c) = \frac{n_{c j v} + \alpha}{n_{c j} + \alpha K_j}$ with $\alpha > 0$. Every level, including one with count 0 in a class, gets a strictly positive probability; e.g. with $n_{c j} = 50$ attack flows, $K_j = 6$ levels and $\alpha = 1$, a level never seen among attacks gets $1/56$ instead of 0.

(e)

- *Discriminative* (logistic regression, trees): with many labelled flows and strongly correlated features, it models $p(y | x)$ directly, does not double-count correlated evidence, and does not spend effort modelling how flows are distributed.
- *Generative* (Naive Bayes, Gaussian models): with few labelled attacks its structured class models are stable; the prior is an explicit parameter, so a change in attack prevalence is corrected by swapping π ; and the class likelihood $p(x | y)$ can flag flows that fit no known class (a reject option) [one-read §4.8].

1.8 Problem 8: Detecting an attack from a sequence of flows

Problem 8

Flows from one source host; flow 1 is benign background traffic and a port scan starts at $a_2 = 17$ s. Flow i starts at a_i , completes at t_i ; its score s_i is available at t_i .

i	1	2	3	4	5	6
a_i (s)	0	17	30	43	56	73
t_i (s)	2	18	31	44	57	74
port	80	22	23	25	445	3389
outcome	success	fail	fail	fail	fail	fail
s_i	0.10	0.18	0.22	0.24	0.28	0.26

Flow-level alert if $s_i \geq 0.70$. Window detector: at each t_i , a 60-s window holds flow i and earlier flows completed after $t_i - 60$; alert if ≥ 4 flows, ≥ 4 distinct ports, failure fraction ≥ 0.75 , and score sum ≥ 0.70 . (a) Which flows trigger the flow-level detector, and what does it show? (b) Window features after each flow until the first alert; detection time; delay from the episode start and from its first flow’s completion. (c) Repeat with a 20-s window; benefit and limitation. (d) What information is available at each decision time; two pieces of future-information leakage. (e) The source IP is a shared NAT address: effect and a better grouping key. (f) $A_i = 0.5A_{i-1} + s_i$, $A_0 = 0$, alert above 0.45: compute, compare, one difficulty. (g) A leakage-safe split when several windows come from one episode.

What this tests

Evidence spread over several inputs; aggregating information that is available at decision time; leakage over time; grouped and temporal splits [one-read §3.6, 4.8].

(a) None: the largest score is $0.28 < 0.70$. Each scan flow on its own is only mildly suspicious, a failed connection to a common port. The attack is a *pattern across flows* (many ports, mostly failures, in a short time), which no per-flow classifier can see because its input contains one flow.

(b) The window at t_i is $(t_i - 60, t_i]$.

after flow	t_i	window	flows in it	# flows	# ports	failure fraction	score sum
1	2	$(-58, 2]$	1	1	1	$0/1 = 0$	0.10
2	18	$(-42, 18]$	1, 2	2	2	$1/2 = 0.50$	0.28
3	31	$(-29, 31]$	1, 2, 3	3	3	$2/3 = 0.67$	0.50
4	44	$(-16, 44]$	1, 2, 3, 4	4	4	$3/4 = 0.75$	0.74

After flow 4 all four conditions hold ($4 \geq 4$, $4 \geq 4$, $0.75 \geq 0.75$, $0.74 \geq 0.70$): **alert at $t = 44$ s.** Delay from the start of the episode: $44 - 17 = 27$ s. Delay from the completion of the episode's first flow: $44 - 18 = 26$ s.

Notice that the benign flow 1 is inside the window and helps reach “4 flows, 4 ports, sum 0.74”. Without it, the window at $t = 44$ would hold only 3 flows; the first alert would come at $t = 57$, with flows 2–5: 4 flows, 4 ports, failure fraction 1, sum 0.92. The window does not know which flows belong to the episode.

(c) With a 20-s window $(t_i - 20, t_i]$:

t_i	2	18	31	44	57	74
window $(t_i - 20, t_i]$	$(-18, 2]$	$(-2, 18]$	$(11, 31]$	$(24, 44]$	$(37, 57]$	$(54, 74]$
flows in window	1	1, 2	2, 3	3, 4	4, 5	5, 6
# flows	1	2	2	2	2	2

The scan flows are about 13 s apart, so a 20-s window never holds more than 2 of them: **no alert**. To contain 4 consecutive scan flows the window must exceed their span, $57 - 18 = 39$ s.

- *Benefit*: a short window reacts quickly to fast bursts and is not diluted by old or unrelated traffic.
- *Limitation*: it misses slow (“low-and-slow”) scans whose flows are spread out, which attackers can do deliberately to stay under such thresholds.

(d) At the decision time t_i the detector may use: the features and score of flow i and of every earlier flow from the same source completed by t_i , and the model and thresholds fixed beforehand. It may not use anything later or anything derived from the label. Two examples of future-information leakage:

1. a centred window $(t_i - 30, t_i + 30]$, or window features computed over the whole episode, which include flows not yet completed at t_i ;
2. using the ground-truth episode boundaries (e.g. starting the window at $a_2 = 17$ because you know the scan starts there), or a normalisation statistic computed over the whole day including later traffic.

(e) Behind a NAT, many devices share one public source IP. A window keyed by that IP mixes their flows:

- benign devices add flows and distinct ports, so busy legitimate traffic can satisfy “ ≥ 4 flows, ≥ 4 ports” (false alarms);
- their successful connections dilute the failure fraction, so a real scan can fall below 0.75 (missed attacks).

A better key identifies the device: the internal (pre-NAT) source address from internal logs, the MAC address or DHCP lease, or an endpoint or session identifier. Equivalently, place the sensor inside the network, before translation.

(f)

i	t_i	$0.5A_{i-1}$	s_i	A_i
1	2	0	0.10	0.100
2	18	0.050	0.18	0.230
3	31	0.115	0.22	0.335
4	44	0.1675	0.24	0.4075
5	57	0.2038	0.28	0.4838 > 0.45

Alert at $t = 57$ s, 13 s later than the 60-s window. The accumulation uses only the scores, not the behavioural features (ports, failures), so it needs more evidence. *One difficulty*: the decay factor and the threshold interact and are hard to choose. A constant stream of scores s converges to $A^* = s/(1 - 0.5) = 2s$, so any benign host whose flows score about 0.23 or more would eventually alert. The decay is also per flow, not per second, so a burst and a slow trickle with the same scores are treated alike. Both are hyperparameters to select on validation data.

(g) Windows built from one episode overlap and share flows, so they are strongly dependent; if some go to training and others to test, the test windows are near-copies of training windows. Split by *episode*: all windows of an episode go to one subset (benign hosts grouped likewise). When drift matters, order the split in time as well (training episodes before validation before test) and leave a gap at least one window long at each boundary, so no window straddles two subsets. Fit any normalisation on training windows only, choose the window length and thresholds on validation, and use the test set once.

2 Exercises for Chapter 1: What does it mean to learn?

These questions have no single correct wording; what matters is the reasoning. Each answer below shows the reasoning a good answer needs, tied to the concepts of Lecture 1 [one-read §1].

2.1 Exercise 1: Write the rules

Exercise 1

A 20×20 binary image contains a handwritten digit; write a conventional program deciding whether it is a “3”. Propose three or four rules, then consider shifting, narrowing, slight rotation and thicker strokes. (a) Which rules still work? (b) How could you modify them? (c) What new difficulties do the modifications introduce? (d) When would you rather provide examples than add rules? What does this reveal?

What this tests

Having an ability is not the same as having an explicit procedure that describes it [one-read §1.1].

Possible rules. (R1) the number of black pixels lies between 40 and 90; (R2) scanning each row from the right, there are two bulges stacked vertically, i.e. the stroke reaches far right twice with a dip between; (R3) the left part of the middle rows is empty (a “3” is open on the left); (R4) there is no closed loop (unlike 0, 6, 8, 9).

(a) *Shift*: R1 and R4 still work; R2 and R3 work only if they are written relative to the digit’s bounding box, not to fixed pixel columns. *Narrower*: R1 may fail (fewer pixels), R2’s bulges get smaller and may disappear. *Slight rotation*: R2 and R3 are defined along rows and columns, so they degrade: the “left side” is no longer the left. *Thicker strokes*: R1 fails (many more pixels), R4 can fail because thick strokes close the gaps of a “3” into loops, and R3 fails when the middle stroke spreads left. Topological rules (R4) survive shifts best; pixel-count and absolute-position rules fail first.

(b) Normalise before applying the rules: centre the digit on its centre of mass, rescale its bounding

box to a fixed size, deskew it (estimate the slant and undo it), and thin the strokes to one-pixel skeletons. Then state rules on the normalised image in relative coordinates.

(c) Each fix adds its own failure modes: deskewing estimates the slant and can be wrong; thinning creates spurious branches (“spurs”) and breaks; every rule now has thresholds that must be tuned; rules interact, so a change that fixes one digit breaks another; and the remaining exceptions never end (a “3” with a flat top, a “3” that touches itself, sloppy writing that resembles a 5 or an 8).

(d) When writing and maintaining rules costs more than collecting labelled examples: when each new batch of handwriting reveals new exceptions, when the rule set has grown so large that changes have unpredictable side effects, and when examples are cheap (many digits, easy to label). *What it reveals*: people recognise a “3” effortlessly without being able to state the procedure they use; the variations are too many to enumerate. Learning from examples lets the system absorb the variations present in the data instead of requiring a human to anticipate each one.

2.2 Exercise 2: The knowledge engineer

Exercise 2

In the 1980s, you build an expert system for diagnosing industrial machine faults, with two hours a week of an experienced engineer’s time. After months it has hundreds of IF–THEN rules. A new generation of machines appears with faults the engineer has never seen. Which difficulties come from obtaining knowledge from the expert, from representing it as rules, and from unfamiliar situations? Why might learning help, and why could expert knowledge still matter?

What this tests

The knowledge acquisition bottleneck, and its three distinct parts [\[one-read §1.1\]](#).

Obtaining knowledge. The expert’s time is scarce (two hours a week); much of their skill is tacit, so they cannot state it as rules and their explanations may be reconstructed after the fact or inconsistent; and for the new machines they have no experience at all to give.

Representing it as rules. Words like “high” and “strong” need thresholds that are fragile; hundreds of rules interact and conflict; IF–THEN rules express certainty poorly; and every addition can break existing behaviour, so maintenance grows with the size of the rule base.

Unfamiliar situations. The rules cover only the faults the expert knew; for new machines and new faults the system is silent or confidently wrong, and it has no mechanism to extend itself.

Why learning helps. If the new machines produce sensor data and fault records, a learning system can extract patterns directly from that experience, including patterns nobody has articulated, and can be retrained as new records accumulate, without waiting for an expert to explain each fault.

Why expert knowledge still matters. The expert knows which measurements are relevant (features such as temperature or vibration spectra), defines the fault categories and labels, checks that learned behaviour is plausible and safe, and supplies constraints when data are scarce. Learning shifts the expert’s role from writing rules to shaping the experience and checking the result.

2.3 Exercise 3: Two collections of photographs

Exercise 3

A system must recognise ten bird species. Dataset A: one million photos from one camera in one wildlife park. Dataset B: fifty thousand photos by many people, in different countries, seasons, weather, backgrounds and angles. Which do you choose, distinguishing amount of experience from its diversity and representativeness? When could the other be preferable?

What this tests

Generalisation depends on whether the experience resembles the situations of use, not only on its quantity [one-read §1.5].

For a system used by the general public, **B**. Its photos cover the variation the system will meet: backgrounds, lighting, seasons, poses, cameras. **A** has twenty times more images but they repeat the same camera, park, backgrounds and probably the same individual birds; the model can learn shortcuts specific to that setting (the park's vegetation, the camera's colour response) that fail elsewhere. *Amount* is how many examples there are; *diversity* is how much of the space of situations they cover; *representativeness* is whether they are distributed like the situations of use. A million near-repeats can carry less useful information than fifty thousand varied images.

A would be preferable if the system is deployed exactly there: a fixed camera trap in that park, where the deployment distribution *is* **A**'s distribution. It could also be preferable if **B**'s labels are much noisier, or if **B** contains very few images of some of the ten species.

2.4 Exercise 4: The mysterious shortcut

Exercise 4

A wolf-versus-dog system performs very well for its developers and much worse after deployment. Most wolf training photos contained snow; most dog photos did not. (a) A plausible explanation for the initial success. (b) Why might the strategy fail on new photos? (c) Two new photographs that would test whether it learned about wolves or about the background. Relate to generalisation.

What this tests

Stable versus spurious regularities; a shortcut fits the training data and fails at deployment [one-read §1.5, 3.1].

(a) In the developers' data, "snow in the background" predicts "wolf" almost perfectly, and snow is a large, easy visual cue (bright, uniform pixels). A system can reach high accuracy by detecting snow without learning anything about wolves; the developers' test images had the same correlation, so the shortcut also scored well there.

(b) The correlation between snow and wolves was created by how the photos were collected, not by what wolves are. In new photos, wolves appear on grass and dogs in snow; a system relying on the background fails on exactly those.

(c) (1) *A dog in snow*: a background-based system says "wolf". (2) *A wolf on grass or in a forest without snow*: it says "dog". If the system gets these right, it uses the animal; if it gets them wrong, it uses the background. *Generalisation*: good performance on new situations requires that the regularity the system learned still holds in those situations; a regularity can explain all the training data and still not be the one that holds at deployment.

2.5 Exercise 5: Same world, different questions

Exercise 5

A hospital has many chest X-rays, some with diagnoses. Projects: (a) predict whether a new X-ray shows pneumonia; (b) group thousands of unlabelled X-rays by visual similarity; (c) remove part of each X-ray and predict the missing region; (d) train a robot to position an imaging sensor, rewarding good positioning and penalising unsafe moves. For each: what experience, what feedback, which paradigm, and why they differ.

What this tests

The three forms of experience and the learning settings they lead to [one-read §1.2, 1.4].

	Experience	Feedback	Paradigm
(a)	X-rays paired with diagnoses	comparison of the prediction with the recorded diagnosis	supervised (classification)
(b)	unlabelled X-rays	none from outside; only an internal criterion (images in a group should be similar)	unsupervised
(c)	unlabelled X-rays	the hidden region itself: the data supply the answer	self-supervised
(d)	interaction: actions and their consequences	rewards and penalties, possibly delayed; actions change what is observed next	reinforcement

They differ in *where the information about doing well comes from*: a human label (a), no external answer at all (b), an answer manufactured from the data (c), or consequences of the learner's own actions (d). Only in (d) does the learner influence its future experience.

2.6 Exercise 6: What exactly is being supervised?

Exercise 6

A university has ten years of records (previous grades, attendance, course choices, eventual outcome) and wants to predict whether first-year students will complete their degree. Objectives: (a) predict whether each student graduates; (b) predict the final graduation grade; (c) discover groups of students with similar trajectories, with no desired outcome. Inputs, outputs, problem type; why can one dataset support several problems?

What this tests

The output decides the problem; the same observations support several problems [one-read §1.3].

	Input	Desired output	Type
(a)	first-year information: prior grades, attendance, course choices	graduated: yes / no	classification
(b)	the same	final grade, a number	regression
(c)	the trajectories	none	neither (unsupervised)

Two cautions a good answer mentions. The input must contain only what is known at prediction time (the first year), not later grades, or the model leaks the outcome. And (b) is only defined for students who graduate, so it is trained on a selected subgroup. One dataset supports several problems because a learning problem is defined by *which variable you choose to predict*, if any, not by the data themselves.

2.7 Exercise 7: The robot that learns in the wrong world

Exercise 7

A warehouse robot trains entirely in simulation: flat floors, identical box friction, constant lighting. It does well in simulation and poorly in the real warehouse. You may change the simulator, not the learning algorithm. What would you vary, and why would a wider range of conditions help? Would more visual realism necessarily solve the problem?

What this tests

Generalisation needs experience that covers the variation of the deployment world [one-read §1.5].

Vary everything that differs in reality and that the policy might depend on: floor slopes and bumps; friction of boxes and floors; box sizes, masses and weight distributions; lighting levels and shadows; sensor noise and delays; motor response; the behaviour of workers and vehicles. A policy trained across many such settings cannot rely on any one simulator constant (it never knows the exact friction), so it must learn behaviour that works across the range; the real warehouse then looks like one more variation it has already handled.

More visual realism does not necessarily help: much of the mismatch is *physical* (friction, flat floors, dynamics), not visual, and a photorealistic simulator with perfectly flat floors still trains the robot for the wrong world. Broad variation in the factors that matter is usually worth more than high fidelity in a single setting.

2.8 Exercise 8: Design the experience

Exercise 8

You must detect phishing websites. Before choosing an algorithm, decide what experience to collect: where examples come from, how varied, how correct outcomes are established, and how fast phishing changes. Why can a very large dataset still produce a poor system?

What this tests

Experience is the first ingredient of a learning system; its choice determines what can be learned [one-read §1.5].

Sources. Phishing: reported URL feeds, email-gateway detections, user reports, crawls of newly registered domains. Legitimate: not only popular sites but the long tail of small legitimate sites, and in particular real login pages of banks and services (the hard negatives that phishing imitates).

Variety. Many targeted brands and languages; different hosting (free hosts, compromised legitimate sites, new domains); different phishing kits; mobile and desktop versions; URL shorteners and redirects.

Correct outcomes. Confirm phishing by analyst verification, takedown records, or agreement of several independent blocklists; record when the label was established; expect some label noise and measure it.

Speed of change. Phishing pages live for hours, and techniques change in weeks, so collection must be continuous, evaluation temporal (train on the past, test on the future), and the model retrained regularly.

Why size is not enough. A huge dataset drawn from one feed, one kit or one month, with legitimate examples taken only from the most popular sites, lets the model learn shortcuts (“new domain”, “unknown site”) rather than what makes a page phishing; it is outdated by deployment; near-duplicate kit pages across training and test inflate the evaluation; and features such as “listed on a blocklist” leak the label. Badly chosen experience at scale teaches the wrong lesson very confidently.

2.9 Exercise 9: Four ingredients, one system

Exercise 9

Choose an application not used in the chapter and describe how you would begin designing it without naming an algorithm: (a) available experience; (b) form of the output; (c) information the algorithm could use to improve; (d) how to decide whether it performs well. Map each part to the four ingredients.

What this tests

Experience, model, learning algorithm, evaluation criterion [one-read §1.5].

Application: forecasting a city's electricity demand for each hour of the next day.

- (a) *Experience*: several years of hourly demand with the weather (temperature, humidity), calendar (weekday, holidays) and special events at the time.
- (b) *Output*: a number for each of the 24 hours, possibly with a range (an upper quantile), since under-supply is costlier than over-supply.
- (c) *Improvement signal*: each day, the difference between predicted and realised demand; the model is adjusted to reduce it on past days.
- (d) *Evaluation*: error on later days not used for training (temporal split), weighted by the asymmetric cost of under- versus over-prediction, and compared with a simple baseline ("same as last week").

Mapping: (a) is the **experience**; (b) is the **model**, the form in which knowledge is represented and output; (c) is the **learning algorithm**, how experience changes the model; (d) is the **evaluation criterion**.

2.10 Exercise 10: A claim worth questioning

Exercise 10

A company claims: "Our AI has learned every possible situation from millions of examples." Explain what is problematic, without mathematical terminology, discussing experience, situations already observed, and situations after deployment.

What this tests

Finite experience, and generalisation as the real goal [one-read §1.5].

However many examples it has seen, a system's experience is finite, while the situations it can meet are not: new people, new places, new conditions, a world that changes over time, and in security an adversary who deliberately does what was never seen before. Millions of examples are still a sample of situations, and often many of them are similar to one another, so their number says little about how much of the world they cover. What the system has really done is perform well on the situations it observed; whether it performs well on the ones it has not is exactly the question of generalisation, and it can only be answered by testing on genuinely new situations. The claim confuses having seen many examples with having seen every case. If a system really had seen every possible situation, it would not need to learn at all; it could just look up the answer. Learning matters precisely because it has not.

3 Exercises for Chapter 2: From representations to decisions

3.1 Exercise 1: Representation and unavoidable ambiguity

Exercise 1

Two different network events z_A (benign) and z_B (malicious) map to the same feature vector $x = g(z_A) = g(z_B)$. (a) Why can no deterministic classifier using only x classify both correctly? (b) Among events represented by x , 30% are malicious. Bayes decision and conditional error under equal costs? (c) Propose an additional feature that might separate them; when is it legitimate and when is it leakage? (d) Can adding irrelevant features leave the Bayes error unchanged while making learning from finite data harder?

What this tests

The representation fixes what information exists; the Bayes error is a property of $P(Y | X)$, not of the learner [one-read §2.1, 2.2].

- (a) A deterministic classifier is a function h of its input. Both events have input x , so both receive the label $h(x)$. Since their true labels differ, h is wrong on one of them whatever it does. The information that would separate them was lost when g mapped two different events to one vector.
- (b) $P(Y = 1 | X = x) = 0.3$ and $P(Y = 0 | X = x) = 0.7$. Under equal costs the Bayes rule picks the more probable class: **benign**. Its conditional error is the probability that the truth is the other class: 0.3. No classifier using only x can do better at this x , because any rule is wrong either on the 30% or on the 70%. This 0.3 belongs to the problem (to the representation g), not to any particular model.
- (c) For example the destination port, the payload size distribution, or the process that opened the connection on the host. It is *legitimate* if it is available, in the same form, at the moment a deployed system must decide, and would be computed the same way for new events. It is *leakage* if it is only known after the fact or is produced by the labelling process, e.g. “matched an IDS signature written after the incident”, “host appears in the incident report”, or a blacklist updated using the attack itself.
- (d) Yes. Let U be an irrelevant feature, meaning that given X it carries no information about Y : $P(Y | X, U) = P(Y | X)$. The Bayes classifier with (X, U) makes exactly the same decisions as with X , so the Bayes error is unchanged. With finite data, however, U hurts: in KNN it adds a coordinate in which neighbours differ randomly, so the nearest neighbours in (X, U) are less similar in X ; in a parametric model it adds parameters to estimate (more variance) and an opportunity to fit spurious correlations. Same achievable error, harder to approach [one-read §2.4, 3.2].

3.2 Exercise 2: The loss changes the meaning of the best prediction

Exercise 2

For a fixed x , $p(y | x) = \frac{1}{4}e^{-y/4}$, $y \geq 0$, with $F(y | x) = 1 - e^{-y/4}$. The prediction is an action $a \geq 0$. (a) Optimal prediction under squared loss and under absolute loss; why do they differ? (b) Asymmetric loss $\ell(y, a) = 4(y - a)$ if $a < y$, and $a - y$ if $a \geq y$: optimal decision and interpretation. (c) Why may a model that outputs only the conditional mean be insufficient for the asymmetric loss? What richer quantity could it estimate?

What this tests

The Bayes rule for regression: the loss decides which summary of $P(Y | x)$ to report [one-read §2.2].

- (a) *Squared loss*. The minimiser of $\mathbb{E}[(Y - a)^2 | x]$ is the conditional mean [one-read §2.2]:

$$\mathbb{E}[Y | x] = \int_0^\infty y \frac{1}{4}e^{-y/4} dy = 4.$$

(An exponential distribution with density $\frac{1}{\beta}e^{-y/\beta}$ has mean β ; here $\beta = 4$.)

Absolute loss. The minimiser of $\mathbb{E}|Y - a|$ is a median: its derivative in a is $F(a) - (1 - F(a))$, zero when $F(a) = \frac{1}{2}$:

$$1 - e^{-a/4} = \frac{1}{2} \iff e^{-a/4} = \frac{1}{2} \iff a = 4 \ln 2 \approx 2.77.$$

Why they differ. The distribution is right-skewed: most response times are short, but a long tail of slow ones pulls the mean up. Squared loss punishes large errors heavily, so it chases the tail and reports the mean; absolute loss treats all errors linearly and reports the middle value, ignoring how far the tail extends.

(b) Write the expected loss as two integrals, under-prediction ($y > a$, cost 4 per unit) and over-prediction ($y \leq a$, cost 1 per unit):

$$\mathbb{E}[\ell(Y, a) \mid x] = 4 \int_a^\infty (y - a) p(y) dy + \int_0^a (a - y) p(y) dy.$$

Differentiate with respect to a (the boundary terms vanish because the integrands are 0 at $y = a$):

$$\frac{d}{da} = -4 \int_a^\infty p(y) dy + \int_0^a p(y) dy = -4(1 - F(a)) + F(a).$$

Setting this to zero gives $F(a) = \frac{4}{5} = 0.8$, the 80th percentile. (The second derivative is $5p(a) > 0$, so it is a minimum.)

$$1 - e^{-a/4} = 0.8 \iff e^{-a/4} = 0.2 \iff a = 4 \ln 5 \approx 6.44.$$

Interpretation. Under-predicting costs four times more than over-predicting (e.g. promising a response time and missing it is worse than being early), so the optimal prediction leans high: it is chosen so that the response exceeds it only 20% of the time. In general the level is $c_u/(c_u + c_o) = 4/5$ [one-read §2.2, mastery box].

(c) Distributions with the same mean can have very different 80th percentiles: a narrow distribution around 4 has its 80th percentile near 4; this skewed one has it at 6.44. Knowing only $\mathbb{E}[Y \mid x]$ does not determine the quantile, so a model trained to output the mean cannot serve the new loss. It should estimate a richer quantity: the conditional distribution $p(y \mid x)$ itself (from which any quantile follows), or directly the needed conditional quantiles (quantile regression, trained with the matching asymmetric loss). If the family is known, e.g. exponential, the mean does determine the quantile ($m \ln 5$), but only because of that extra distributional assumption.

3.3 Exercise 3: Classification with a reject option

Exercise 3

$p(x) = P(Y = 1 \mid X = x)$, class 1 = attack. Actions: attack, benign, defer to a human. False positive costs 1, false negative 9, deferral always 0.6; correct automatic decisions cost 0. (a) Conditional risk of each action. (b) Intervals of p where each is optimal. (c) Why is one threshold no longer sufficient? (d) How do the regions change if review becomes more expensive?

What this tests

Bayes decision with more than two actions: lower envelope of the risk lines [one-read §2.2]. Same structure as Tutorial Problem 2(c).

(a)

$$\text{declare attack} \quad R(\text{attack} \mid x) = 1 \cdot (1 - p),$$

$$\text{declare benign} \quad R(\text{benign} \mid x) = 9p,$$

$$\text{defer} \quad R(\text{defer} \mid x) = 0.6.$$

(b) Benign is optimal when $9p \leq 0.6$ and $9p \leq 1 - p$, i.e. $p \leq 1/15 \approx 0.067$ and $p \leq 0.1$, so $p \leq 0.067$. Attack is optimal when $1 - p \leq 0.6$ and $1 - p \leq 9p$, i.e. $p \geq 0.4$ and $p \geq 0.1$, so $p \geq 0.4$. Defer is optimal in between:

$$p \leq 0.067 : \text{benign}, \quad 0.067 < p < 0.4 : \text{defer}, \quad p \geq 0.4 : \text{attack}.$$

(c) Two actions give two risk lines that cross once, so a single threshold ($p = 0.1$ here) separates them. With a third action of constant cost, the cheapest action changes *twice* as p increases (benign,

then defer, then attack), so the decision needs two thresholds. Geometrically, the horizontal line 0.6 cuts across the peak of the lower envelope $\min\{9p, 1 - p\}$, whose maximum is 0.9 at $p = 0.1$.

(d) With deferral cost C_R the band is $C_R/9 < p < 1 - C_R$. As C_R grows the lower boundary moves up and the upper boundary moves down, so the band shrinks. When $C_R \geq 0.9$ (the peak of the envelope) deferral is never optimal and the single threshold $p = 0.1$ returns. Cheaper review widens the band: more cases go to humans.

3.4 Exercise 4: Training loss and decision performance can disagree

Exercise 4

Two binary classifiers; probability assigned to the correct class on four examples: Model A 0.60, 0.60, 0.60, 0.60; Model B 0.99, 0.99, 0.99, 0.20. (a) Compare with zero-one loss at threshold 0.5. (b) Compare average cross-entropy. (c) Why can the two criteria rank them differently? (d) Does this show cross-entropy is an inappropriate training loss?

What this tests

Training loss versus decision loss [one-read §2.6]; cross-entropy rewards good probabilities, not only correct sides of 0.5 [one-read §2.5].

(a) An example is classified correctly when the probability of its true class exceeds 0.5. A: all four correct, error 0. B: three correct, the fourth (0.20) wrong, error $1/4 = 0.25$. **A is better.**

(b)

A: $-\ln 0.6 = 0.511$ on each example, so the average is 0.511;

$$\text{B: } \frac{3 \times (-\ln 0.99) + (-\ln 0.20)}{4} = \frac{3 \times 0.0101 + 1.609}{4} = 0.410.$$

B is better (lower).

(c) Zero-one loss only asks which side of 0.5 each probability falls on; it cannot tell 0.51 from 0.99. Cross-entropy measures how much probability was put on the truth, on a log scale: B is very confident and right three times, and its one mistake (0.20) is costly but not catastrophic; A is barely right everywhere. They measure different things, so they can disagree.

(d) No. A careful argument has three steps.

1. The two losses play different roles. The training loss is used to *estimate* the model; the decision loss is applied *afterwards* to the estimated probabilities [one-read §2.6].
2. Cross-entropy is the right loss for estimating probabilities: its expected value is minimised exactly when the predicted probabilities equal the true conditional probabilities, since $H(p, \hat{p}) = H(p) + D_{KL}(p \| \hat{p})$ [one-read §2.5]. Zero-one loss cannot be used for this: it is flat almost everywhere (zero gradient) and is indifferent to how confident the model is.
3. If the probabilities are close to the truth, the Bayes rule computed from them is close to optimal for *any* decision loss: zero-one, cost-weighted, or with a reject option. So learning good probabilities is the right training goal even when the final criterion is zero-one.

Four examples are far too few to say which model has lower population zero-one error. And A's flat 0.6 everywhere would be useless with unequal costs or a reject option, where the actual size of the probability matters.

3.5 Exercise 5: A nearest neighbour created by the units

Exercise 5

Query: income 50,000, age 30, $x = (50,000, 30)$. $x_A = (51,000, 60)$, $x_B = (60,000, 31)$. (a) Nearest with raw Euclidean distance. (b) Scale income by 10,000 and age by 10; nearest again. (c) What changed mathematically, and what did not change about the people? (d) Location one-hot encoded: what does Euclidean distance assume about locations? Propose a domain-aware alternative.

What this tests

Distances depend on units and encodings [one-read §2.4].

(a) $d_A = \sqrt{1000^2 + 30^2} = \sqrt{1,000,900} \approx 1000.4$; $d_B = \sqrt{10000^2 + 1^2} \approx 10000.0$. **A is nearest**: a person 30 years older, because income in raw units swamps age.

(b) Scaled: $x = (5, 3)$, $x_A = (5.1, 6)$, $x_B = (6, 3.1)$. $d_A = \sqrt{0.01 + 9} \approx 3.00$; $d_B = \sqrt{1 + 0.01} \approx 1.00$. **B is nearest**.

(c) Dividing coordinate j by s_j is the same as using the weighted distance $\sqrt{\sum_j (x_j - z_j)^2 / s_j^2}$ on the original data: the scaling changed the metric, i.e. how much a unit of each feature counts. It did not change anything about the people: same incomes, same ages. “Nearest” is a property of the representation and the distance, not of the individuals; that is why the scaling must be chosen deliberately (standardisation is the default, with means and standard deviations fitted on the training set) [one-read §2.4, 3.6].

(d) With one-hot encoding every location is a unit vector on its own axis, so any two different locations are at distance $\sqrt{2}$: Euclidean distance assumes all locations are *equally* different (Rabat is as far from Salé as from Paris), and the size of that difference relative to other features is set by the encoding rather than by meaning. Alternatives: represent location by coordinates (latitude, longitude) or travel distance; use a hierarchy (same neighbourhood < same city < same region < same country) with a specified distance table; or learn an embedding of locations from data; and weight the location block against the other features, choosing the weight by validation.

3.6 Exercise 6: Local neighbourhoods in high dimension

Exercise 6

n points uniform in the unit d -ball; query at the centre; a useful neighbour lies within distance $r = 0.1$; M_d = number of such points. (a) Distribution of M_d , its expectation, and $P(M_d \geq 1)$. (b) For $n = 1000$, first d with $P(M_d \geq 1) < 0.5$. (c) Radius r_d giving $P(M_d \geq 1) = 0.95$; behaviour as d grows. (d) Which assumptions are favourable to KNN (query location, uniformity, irrelevant dimensions)?

What this tests

The curse of dimensionality as a probability calculation [one-read §2.4, mastery box].

(a) One point falls in the small ball of radius r around the centre with probability equal to the volume ratio. The volume of a d -ball of radius r is r^d times the volume of the unit ball, so

$$P(\text{one point within } r) = r^d = 0.1^d.$$

The n points are independent, each landing inside with probability 0.1^d , so M_d counts successes in n independent trials:

$$M_d \sim \text{Binomial}(n, 0.1^d), \quad \mathbb{E}M_d = n 0.1^d, \quad P(M_d \geq 1) = 1 - P(M_d = 0) = 1 - (1 - 0.1^d)^n.$$

(b) With $n = 1000$ (using $(1 - q)^n \approx e^{-nq}$ for small q):

d	1	2	3	4
0.1^d	0.1	0.01	0.001	0.0001
$\mathbb{E}M_d$	100	10	1	0.1
$P(M_d \geq 1)$	≈ 1	$1 - e^{-10.05} \approx 1.000$	$1 - 0.999^{1000} \approx 0.632$	$1 - e^{-0.1} \approx 0.095$

The first dimension with $P(M_d \geq 1) < 0.5$ is $\mathbf{d} = 4$: with a thousand points, there is a 90% chance that not a single one lies within 0.1 of the query.

(c) Solve $1 - (1 - r^d)^n = 0.95$: $(1 - r^d)^n = 0.05$, so $r^d = 1 - 0.05^{1/n}$ and

$$r_d = (1 - 0.05^{1/n})^{1/d} \approx \left(\frac{\ln 20}{n}\right)^{1/d} \approx \left(\frac{3}{n}\right)^{1/d}.$$

For $n = 1000$, $r^d = 0.00299$:

d	1	2	3	5	10	20	100
r_d	0.003	0.055	0.144	0.313	0.559	0.748	0.943

As d grows, $r_d \rightarrow 1$: to be fairly sure of finding even one neighbour, the “neighbourhood” must cover most of the ball. The neighbours are then no longer local, and averaging their labels no longer estimates the conditional behaviour at the query.

(d) Several assumptions make this calculation optimistic for KNN:

- *Query at the centre.* A query near the edge of the data has part of its neighbourhood outside the ball, so fewer neighbours. In high dimension almost all the volume is near the surface (the fraction within radius $1 - \varepsilon$ is $(1 - \varepsilon)^d \rightarrow 0$), so typical queries are near the edge.
- *One neighbour is enough.* KNN with $k > 1$ needs k neighbours to average away label noise, which requires a larger radius still.
- *Close means useful.* The calculation assumes every point within r has a similar label. If some dimensions are irrelevant, points close in the relevant coordinates can be far in the irrelevant ones and be excluded, and points that are close overall may differ where it matters: the budget is spent on dimensions that carry no information.
- *Uniformity* cuts both ways: real data often lie near a lower-dimensional structure, which helps KNN (the effective dimension is smaller), but queries may fall in sparse regions, which hurts.

3.7 Exercise 7: Perfect interpolation does not determine behaviour elsewhere

Exercise 7

Training data $(-1, 1), (0, 0), (1, 1)$. Consider $f_1(x) = x^2$ and $f_{2,\alpha}(x) = x^2 + \alpha(x+1)x(x-1)$, $\alpha \in \mathbb{R}$. (a) Show all have zero squared training error. (b) Compare predictions at $x = 1/2$. (c) Why can ERM not choose among them? (d) What extra assumptions are introduced by preferring the quadratic, regularising, or using validation data?

What this tests

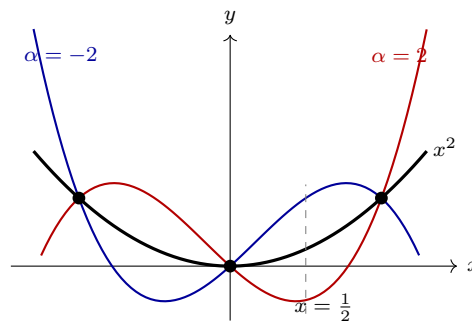
ERM can have infinitely many minimisers; inductive bias chooses among them [one-read §2.6, 3.1].

(a) f_1 gives 1, 0, 1 at $x = -1, 0, 1$: exact. The extra term $\alpha(x+1)x(x-1)$ is a product that contains the factor $(x+1)$, x or $(x-1)$, so it is 0 at each of the three training inputs. Hence $f_{2,\alpha}(x^{(i)}) = f_1(x^{(i)}) = y^{(i)}$ for every i and every α : zero squared training error.

(b) At $x = \frac{1}{2}$: $f_1 = \frac{1}{4}$, and $(x+1)x(x-1) = \frac{3}{2} \cdot \frac{1}{2} \cdot (-\frac{1}{2}) = -\frac{3}{8}$, so

$$f_{2,\alpha}\left(\frac{1}{2}\right) = \frac{1}{4} - \frac{3}{8}\alpha.$$

Every real number is reached by some α : e.g. $\alpha = 2$ gives $-\frac{1}{2}$, $\alpha = -2$ gives 1.



Three of the infinitely many functions that pass through all three training points.

(c) ERM ranks candidates only by training loss. All these functions achieve the minimum possible, 0, so ERM declares a tie among infinitely many minimisers. The training data say nothing about the function between or beyond the three points; any choice is made by something other than the data.

(d)

- *Preferring the quadratic* is an assumption on the hypothesis space: restricting to degree ≤ 2 leaves exactly one interpolant through three points, x^2 . You assume the truth is simple enough to be quadratic.
- *Regularisation* is an assumption that small coefficients are more plausible. In the cubic family $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$, the interpolants are $x^2 + \alpha(x^3 - x)$, with coefficients $(0, -\alpha, 1, \alpha)$ and $\|\theta\|_2^2 = 1 + 2\alpha^2$; the penalty picks $\alpha = 0$, i.e. x^2 .
- *Validation data* is not an assumption about the function but new evidence: points at other x reveal which candidate predicts better. It assumes the validation points are representative of where the model will be used.

3.8 Exercise 8: Are parameters and hyperparameters intrinsic labels?

Exercise 8

System 1: polynomial degree p selected by validation, coefficients $\theta_0, \dots, \theta_p$ fitted on training data. System 2: always degree 20, with an L_1 penalty that makes most coefficients zero. (a) Parameters and hyperparameters of each. (b) How can both produce the same effective degree by different procedures? (c) Is “polynomial degree” always a hyperparameter? (d) What information must stay outside parameter fitting to compare the two fairly?

What this tests

Parameter versus hyperparameter is about how a quantity is determined, not what it is [one-read §2.5]; regularisation as capacity control [one-read §3.4].

(a)

- System 1: parameters $\theta_0, \dots, \theta_p$ (fitted on training data); hyperparameter p (chosen on validation data, and it defines the hypothesis space).
- System 2: parameters $\theta_0, \dots, \theta_{20}$; hyperparameter λ , the strength of the L_1 penalty (chosen on validation). The degree 20 is a fixed design choice, a hyperparameter that is not tuned.

(b) System 1 with $p = 3$ fits a cubic. System 2 with a suitable λ may set $\theta_4, \dots, \theta_{20}$ to exactly zero, which also yields a cubic. One controls capacity by explicit model selection, the other by embedded sparsity inside the fit. The results need not be identical: L_1 also shrinks the surviving coefficients, and the coefficients it keeps need not be the low-order ones (it could keep x^7 and drop x^2).

(c) No. In System 1 the degree is fixed *before* fitting and chosen with validation data: a hyperparameter. In System 2 the effective degree (the highest non-zero coefficient) is an *output* of the parameter fit for a given λ : it is determined by the training data through the objective. Whether a quantity is a parameter or a hyperparameter depends on the procedure: is it set by minimising the training objective, or chosen outside it on held-out data?

(d) Each system used validation data to choose its hyperparameter (p or λ), so that validation set is now optimistic for both. A fair comparison needs data that influenced neither the parameter fits nor the hyperparameter choices: an untouched test set (or nested cross-validation), with both procedures run on the same training rows and evaluated on the same test rows.

3.9 Exercise 9: Several empirical-risk minimisers

Exercise 9

The input population has regions A , B , C . Classifiers h_1 , h_2 agree on A and B and disagree on C . The training sample has no observations from C , although $P(X \in C) > 0$. (a) Why can training loss not distinguish h_1 from h_2 ? (b) Does the tie imply equal population risk? Construct two distributions. (c) What role does the algorithm's tie-breaking rule play? (d) Three ways to obtain evidence about region C .

What this tests

Training data only constrain behaviour where they exist; the rest is decided by inductive bias [one-read §3.1].

(a) The training loss is a sum over training examples, all of which lie in A or B . On those inputs h_1 and h_2 make identical predictions, so every term, and hence the total, is identical.

(b) No. Population risk sums the error over all regions, weighted by probability:

$$R(h) = P(A) \text{err}_A(h) + P(B) \text{err}_B(h) + P(C) \text{err}_C(h).$$

The first two terms are equal for h_1 and h_2 ; the third need not be. Take $P(X \in C) = 0.2$ and let the two distributions agree on A and B (so either could have produced the training sample):

- *Distribution 1*: in C , the true label is always $h_1(x)$. Then $\text{err}_C(h_1) = 0$, $\text{err}_C(h_2) = 1$, and $R(h_2) - R(h_1) = 0.2$.
- *Distribution 2*: in C , the true label is always $h_2(x)$. Now $R(h_1) - R(h_2) = 0.2$.

The training data are equally likely under both, so they cannot tell which world we are in. (A sample with no C points is plausible when $P(C)$ is small: with $P(C) = 0.01$ and $N = 100$, $0.99^{100} \approx 0.37$.)

(c) Some rule picks one of the tied minimisers: the first found, the one closest to the initialisation, the one with the smallest norm, whatever gradient descent converges to. That rule, not the data, decides the behaviour on C . It is an inductive bias, usually implicit, and it should be recognised as such.

(d)

- collect and label data from region C (targeted collection, or ask experts to label a sample);
- use domain knowledge or constraints about C (physical laws, symmetries, known mechanisms) to restrict the hypothesis space;
- hold out validation data that does cover C (from another source or period) to test both classifiers there;
- at deployment, detect inputs from C and abstain or send them to review, monitoring performance there (the reject option, [one-read §4.8]).

3.10 Exercise 10: Audit a misleading security experiment

Exercise 10

Each attack produces hundreds of near-duplicate flows. The team standardises before splitting, assigns rows randomly to training and test, tries 500 configurations, reports the best test accuracy, and uses threshold 0.5 although a missed attack costs 100 times a false alarm. (a) At least four reasons the reported accuracy misleads. (b) Redesign the split and preprocessing. (c) Separate training loss, validation criterion, test metric, deployment decision loss. (d) What final results would let an analyst judge the system responsibly?

What this tests

The complete evaluation chain, combining Tutorial Problems 2 and 4 [one-read §2.7, 3.5, 3.6].

(a)

1. Near-duplicate flows split at random: the test set contains copies of training flows, so the score measures recognition of seen attacks.
2. Standardising before splitting: test statistics leak into the representation.
3. 500 configurations chosen by test accuracy: the test set is a validation set, and the best of 500 noisy scores is strongly optimistic.
4. Accuracy on a rare positive class: it can be high while most attacks are missed.
5. Threshold 0.5 ignores the costs: with $C_{FN} = 100 C_{FP}$ the Bayes threshold is $C_{FP}/(C_{FP} + C_{FN}) = 1/101 \approx 0.01$. At 0.5 many attacks with posterior between 0.01 and 0.5 are left unflagged, so the deployed behaviour is far from the cost-optimal one.
6. No test of novelty or drift: random splitting says nothing about new episodes or later traffic.

(b) Deduplicate; split by attack episode (and benign traffic by host or time block), in time order if drift matters; keep an untouched test set; inside the development data use grouped K -fold cross-validation; put standardisation, imputation and feature selection in a pipeline fitted on the training folds only; choose the configuration *and* the threshold on validation; evaluate once on the test set.

(c)

Training loss	cross-entropy: estimates the probabilities $\hat{p}(y = 1 x)$.
Validation criterion	expected cost per flow with $C_{FP} = 1$, $C_{FN} = 100$ at the candidate threshold (or recall at a fixed false-alarm budget): used to choose the configuration and the threshold.
Test metric	the same criterion, plus recall, precision and false-positive rate, measured once on untouched data.
Deployment decision loss	$C_{FP} = 1$, $C_{FN} = 100$: alert when the (calibrated, prior-corrected) posterior exceeds $1/101$, or the threshold chosen on validation.

(d) The confusion matrix at the operating threshold; recall with its uncertainty (it rests on the number of attacks, not the number of flows, [one-read §3.1]); precision; false-positive rate and false alarms per 100,000 benign flows (or per day); expected cost per flow; recall per attack type, including types held out of training; performance over time; comparison with simple baselines (all-benign, a rule); and a description of the split and of how many configurations were tried.

4 Review questions for Lecture 3: From fitting to generalisation

What this tests

All twenty questions are about one idea: a fitted model is judged on data that did not choose it, and everything (capacity, regularisation, splits, preprocessing) is organised around keeping that true. Sections 3.1–3.6 of the one-read document follow the same order.

Q1. Distinguish empirical risk, population risk and the generalisation gap. Why is only one directly observable during training?

$$\begin{aligned}
 \text{empirical risk} \quad \hat{R}_{\mathcal{D}}(f) &= \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, f(x^{(i)})) \quad (\text{average loss on the sample}) \\
 \text{population risk} \quad R(f) &= \mathbb{E}_{(X,Y) \sim P}[\ell(Y, f(X))] \quad (\text{average loss over all future cases}) \\
 \text{generalisation gap} \quad G(\hat{f}_{\mathcal{D}}) &= R(\hat{f}_{\mathcal{D}}) - \hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})
 \end{aligned}$$

Only the empirical risk can be computed: it needs the data and the model. The population risk is an expectation under the unknown distribution P , and the gap contains the population risk. Held-out data give estimates of R , never R itself.

Q2. Why does evaluating a fitted model on the observations that selected it give an overly favourable result?

The fitting procedure chose $\hat{f}$ precisely to make the loss on these observations small, including by exploiting their accidental regularities (noise, coincidences) that will not recur. Evaluating on the same observations rewards that exploitation. Formally, with $\theta_{\mathcal{H}}^*$ the best parameter in the family,

$$\mathbb{E}_{\mathcal{D}}[\hat{R}_{\mathcal{D}}(\hat{\theta}_{\mathcal{D}})] \leq R(\theta_{\mathcal{H}}^*) \leq \mathbb{E}_{\mathcal{D}}[R(\hat{\theta}_{\mathcal{D}})],$$

so on average the training score is below even the best achievable risk [one-read §3.1].

Q3. One stable and one potentially spurious cue in a cybersecurity application; how could data collection reveal the difference?

Stable: for port scans, the number of distinct destination ports contacted per minute with a high failure rate; it follows from what a scan is. *Spurious:* the subnet or IP range of the attacking machine in the testbed, the capture time, or a TTL value specific to the attack virtual machine; they predict the labels only because of how the dataset was produced. To tell them apart, collect attacks from other hosts, subnets, operating systems and times of day, collect benign traffic at the times attacks happened, and evaluate with grouped or temporal splits: a spurious cue breaks when its correlation with the label is broken.

Q4. What is inductive bias? Through which parts of a learning system can it enter?

A finite sample is consistent with many functions that agree on the sample and differ elsewhere, so learning needs assumptions that prefer some solutions to others; those assumptions are the inductive bias. They enter through (1) the data and representation (which cues exist), (2) the hypothesis space (which relationships can be expressed), (3) regularisation (which fits are preferred), and (4) the optimisation algorithm (which of several minimisers it reaches) [one-read §3.1].

Q5. Why can the minimum training error only decrease when moving to a nested, higher-capacity hypothesis space?

If $\mathcal{H}_m \subseteq \mathcal{H}_{m+1}$, the minimiser of the training error in $\mathcal{H}_m$ is also a candidate in $\mathcal{H}_{m+1}$, so the minimum over the larger set is at most its error: $\min_{\mathcal{H}_{m+1}} \hat{R}_{\text{train}} \leq \min_{\mathcal{H}_m} \hat{R}_{\text{train}}$. A minimum over a superset can never be larger.

Q6. Explain underfitting and overfitting using both training and validation performance.

Underfitting: training error and validation error are both high and close together; the model is too restrictive to capture the structure, so even the training data are fitted poorly. *Overfitting:* training error is low and validation error is much higher; the model has fitted features of the particular sample

that do not carry over. The diagnostic is the pair, and the gap between them, not either number alone.

Q7. *Why does increasing k usually reduce the capacity of KNN?*

The prediction at x averages k labels. With $k = 1$ each training point decides its own neighbourhood: the rule can follow every point, and training error is 0 (barring duplicate inputs with different labels). With larger k each prediction averages over more points, so the fitted function changes more slowly and cannot follow individual observations. At $k = N$ it predicts the global average (or majority class) everywhere: a constant, the lowest capacity. Roughly, KNN has about N/k independent “pieces”.

Q8. *Define statistical bias and variance at a fixed input. How do the definitions use repeated hypothetical training samples?*

Imagine drawing many training sets $\mathcal{D}_1, \mathcal{D}_2, \dots$ from the same population and running the same procedure on each; at a fixed x this gives a spread of predictions $\hat{f}_{\mathcal{D}_1}(x), \hat{f}_{\mathcal{D}_2}(x), \dots$ with mean $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$.

$$\text{Bias}(x) = \bar{f}(x) - f^*(x), \quad \text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2],$$

with $f^*(x) = \mathbb{E}[Y \mid X = x]$. Bias is how far the *centre* of the spread is from the truth; variance is how *wide* the spread is. Neither can be computed from one dataset; they describe the procedure.

Q9. *Identify the three terms in the squared-error bias–variance decomposition. Which cannot be removed even by the optimal predictor?*

$$\mathbb{E}_{\mathcal{D}, Y|x}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x),$$

noise, squared bias, variance. The noise $\sigma^2(x) = \text{Var}(Y \mid X = x)$ is irreducible: even f^* makes this error, because X does not determine Y . (Derivation: [\[one-read §3.3, mastery box\]](#).)

Q10. *What is the interpolation threshold in double descent? Why is the classical U-shaped curve still useful but not universal?*

The interpolation threshold is the smallest capacity at which the model can fit the training sample exactly (training error 0). Validation error can peak there and then decrease again as capacity grows further, the second descent. The U-shape remains a correct description of the classical regime and a good first diagnostic for choosing capacity; but in overparameterised models the curve can go down again, depending on regularisation, the optimiser and which interpolating solution is selected, and the same can happen over training time. So validation behaviour must be examined, not assumed [\[one-read §3.3, figure\]](#).

Q11. *How can a parameter penalty reduce effective capacity without changing the number of parameters?*

The penalised problem $\min_{\theta} \hat{R}_{\text{train}}(\theta) + \lambda \Omega(\theta)$ behaves like fitting within the constrained set $\Theta_c = \{\theta : \Omega(\theta) \leq c\}$, with larger λ corresponding to smaller c . Since $c_1 < c_2$ implies $\Theta_{c_1} \subseteq \Theta_{c_2}$ and $\mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$, a stronger penalty restricts the fit to a smaller family of functions. The parameter vector has the same length; fewer of its values are effectively reachable.

Q12. *Compare L_1 and L_2 regularisation. Why can L_1 produce sparse solutions?*

L_2 , $\Omega = \sum_j \theta_j^2$, shrinks all coefficients smoothly toward zero and usually keeps every variable. L_1 , $\Omega = \sum_j |\theta_j|$, has a constraint region (a diamond in two dimensions) with corners on the coordinate axes; the level sets of the training loss usually touch it first at a corner, where some coefficients are exactly zero: built-in feature selection, unstable when features are correlated [\[one-read §3.4, figure\]](#).

A one-line calculation that shows it (not in the notes, but the cleanest proof). In one dimension, minimise $\frac{1}{2}(\theta - z)^2 + \lambda|\theta|$. For $\theta > 0$ the derivative is $\theta - z + \lambda$, zero at $\theta = z - \lambda$, valid only if $z > \lambda$; symmetrically $\theta = z + \lambda$ if $z < -\lambda$; and if $|z| \leq \lambda$ the minimum is at exactly $\theta = 0$. So L_1 sets every small coefficient to zero (“soft thresholding”). With L_2 , $\frac{1}{2}(\theta - z)^2 + \lambda\theta^2$ gives $\theta = z/(1 + 2\lambda)$, which is zero only when $z = 0$: shrinkage, never exact zeros.

Q13. *Why is early stopping a form of implicit regularisation? What role does patience play?*

Iterative optimisation starts from an initial θ_0 and moves step by step; stopping after t steps limits how far the parameters can travel, so only part of the parameter space is reachable, a smaller effective family. That is regularisation without a penalty term: training time controls capacity. Rule: keep $t^* = \arg \min_t \hat{R}_{\text{val}}(\hat{\theta}_t)$. Because validation loss is noisy, *patience* p lets training continue for p epochs without improvement before stopping (keeping the best checkpoint): too small stops on a random fluctuation, or before an epoch-wise second descent; too large wastes computation.

Q14. *Why is optimal subset selection combinatorial? One strength and one limitation of correlation filtering, forward selection and backward elimination.*

With d features there are 2^d subsets (each feature in or out), and comparing them requires fitting a model for each: $d = 30$ already gives over a billion.

Method	Strength	Limitation
Correlation filter $ \text{Corr}(X_j, Y) $	very fast, model-independent	scores features one at a time: misses nonlinear effects and interactions, ignores redundancy
Forward selection	cheap; each step uses a real validation gain, so some joint effects are seen	greedy: an early choice cannot be undone
Backward elimination	starts from the full model, so it sees all features jointly	expensive in high dimension, needs the full model to be fittable; also greedy

Q15. *Distinguish the roles of training, validation and test data. Why can validation data also be overused?*

Training data fit the parameters and all data-dependent preprocessing; validation data compare configurations and choose features, hyperparameters and thresholds; test data estimate the performance of the final, frozen procedure, once. Validation is overused when many configurations are compared on it, or results are inspected repeatedly to guide the next design: the selected configuration is the one whose noise happened to be favourable, so its validation score is optimistic, $\mathbb{E}[\min_j \hat{R}_{\text{val}}(\hat{f}_j)] \leq \min_j R(\hat{f}_j)$ [one-read §3.1]. The test set protects against this only if its results are never fed back into development.

Q16. *Explain K -fold cross-validation. Why must preprocessing and feature selection be repeated inside each fold?*

Split the development data into K folds; for each configuration, train K times, each time on $K - 1$ folds, and evaluate on the remaining fold; average the K validation risks: $\hat{R}_{CV}(h) = \frac{1}{K} \sum_j \hat{R}_{\text{val}}^{(j)}(\hat{f}_h^{(-j)})$. Each example validates once and trains $K - 1$ times. Preprocessing and feature selection are part of the fitted model: if a scaler or selector were fitted on all development data, the held-out fold would already have influenced the representation used to predict it, and the validation risk would be optimistic. So each fold fits them on its own training part.

Q17. *An application where a random split is reasonable, one requiring a grouped split, one requiring a temporal split.*

Random: quality inspection of items from a stable production line, each item photographed once (new items are like independent draws). *Grouped*: medical images with several scans per patient (group

by patient; the model must work on new patients), or flows grouped by attack episode. *Temporal*: fraud or intrusion detection, demand forecasting: the model is used on the future, and behaviour drifts.

Q18. *Construct an example where accuracy misleads under class imbalance. What questions do precision, recall and F_1 answer?*

1,000 flows, 10 attacks; “always benign” scores 990/1000 = 99% accuracy and catches nothing. *Recall*, $TP/(TP + FN)$: of the real attacks, how many did we catch? *Precision*, $TP/(TP + FP)$: of our alarms, how many were real? $F_1 = 2PR/(P + R)$: a single number that is high only if both are high (harmonic mean); it ignores true negatives and weighs the two errors equally.

Q19. *What does it mean to fit a scaler? Why must its estimated quantities come only from training data?*

Fitting a standardiser means estimating, for each feature j , the mean and variance $\hat{\mu}_j = \frac{1}{N_{tr}} \sum_{i \in tr} x_j^{(i)}$ and $\hat{\sigma}_j^2 = \frac{1}{N_{tr}} \sum_{i \in tr} (x_j^{(i)} - \hat{\mu}_j)^2$, then transforming $z_j = (x_j - \hat{\mu}_j)/\hat{\sigma}_j$. These are learned quantities, like parameters. Computing them on validation or test data lets those data shape the representation (leakage), and at deployment the statistics of future data are not available anyway. The training estimates are reused unchanged everywhere else.

Q20. *How can learning curves distinguish underfitting, high variance and a data-limited regime?*

Plot training and validation error against the training-set size N .

- Both high and close, flattening: **underfitting**; more data will not help, change the representation or add capacity.
- Training low, validation much higher, gap slowly closing: **high variance**; regularise, reduce capacity, or add representative data.
- Both close and still decreasing as N grows: **data-limited**; more representative examples are likely to help.